BÁO CÁO BÀI TẬP LỚN (PBL3)

ĐỊNH TUYẾN ĐỘNG RIPv2 VÀ MÁY CHỦ DHCP

TRIỂN KHAI TRÊN GNS3 VỚI FRROUTING VÀ DOCKER

Người thực hiện:	**Thái Viết Đại**
	Phan Văn Trà
Giảng viên hướng dẫn:	**TS. Đào Duy Tuấn**

Hình thức:	Báo cáo thực hành mạng máy tính
Môi trường:	GNS3 3.0.6, FRRouting 8.4, Docker, VPCS 0.8.5

Báo cáo được xây dựng dựa trên đề bài và bố cục của báo cáo PBL3 gốc trên Cisco Packet Tracer, sau đó tái triển khai và kiểm thử trên GNS3.

Tháng 10 năm 2026

Tóm tắt

Báo cáo trình bày quá trình tái triển khai bài lab PBL3 “Định tuyến động RIPv2 và máy chủ DHCP” trên nền tảng **GNS3 3.0.6**. Thiết kế mạng giữ nguyên theo báo cáo gốc: bốn router R0–R3 kết nối thành vòng, bốn mạng LAN “A”, “B”, “C”, “D” tương ứng với bốn switch, và một DHCP server đặt tại Network A. Thay vì dùng Cisco IOSv cần image có bản quyền, các router được triển khai bằng **FRRouting 8.4** chạy trong container Docker; DHCP server dùng Alpine Linux với BusyBox `udhcpd`; các máy trạm dùng VPCS 0.8.5.

Toàn bộ topology, địa chỉ IP và cấu hình được tạo tự động qua GNS3 API v3 bằng Python. Sau khi RIPv2 hội tụ, cả bốn router đều học đủ ba mạng LAN còn lại; PC_A nhận địa chỉ động trong dải `192.168.10.10--192.168.10.100`; các phép ping liên mạng từ PC_A và PC_D đều thành công. Bộ kiểm thử tự động `test.sh` ghi nhận **9/9** hạng mục PASS. Báo cáo cũng phân tích các vấn đề kỹ thuật gặp phải khi chuyển từ Packet Tracer sang GNS3/FRR.

Mục lục

Tóm tắt **1**

I TỔNG QUAN ĐỀ BÀI VÀ MỤC TIÊU **4**

1.1 Đề bài 4
1.2 Mục tiêu bài lab 4
1.3 Phạm vi và lựa chọn triển khai 4

II SƠ ĐỒ MẠNG VÀ BẢNG PHÂN BỔ ĐỊA CHỈ IP **5**

2.1 Sơ đồ mạng theo báo cáo gốc 5
2.2 Sơ đồ triển khai trên GNS3 5
2.3 Bảng liên kết WAN 6
2.4 Bảng phân bổ địa chỉ IP 6
2.5 Dải địa chỉ DHCP 7

III CÔNG NGHỆ VÀ KIẾN TRÚC TRIỂN KHAI **7**

3.1 GNS3 và API v3 7
3.2 FRRouting trong Docker 8
3.3 DHCP server 8
3.4 Máy trạm VPCS 8
3.5 Kiến trúc tự động hóa 8

IV NỘI DUNG CẤU HÌNH **9**

4.1 Cấu hình FRR cho R0 9
4.2 Tóm tắt cấu hình R1, R2, R3 10
4.3 Cấu hình DHCP server 10
4.4 Cấu hình máy trạm VPCS 10
4.5 Một số điểm kỹ thuật khi tự động hóa 11

V KẾT QUẢ THỰC NGHIỆM **11**

5.1 Môi trường thử nghiệm 11
5.2 Kiểm tra tiền đề 12
5.3 Khởi tạo topology 12
5.4 Giao diện GNS3 13
5.5 Container của lab 14
5.6 Bảng định tuyến RIPv2 15
5.7 Trạng thái tiến trình RIP 15
5.8 Kiểm tra DHCP 16
5.9 Kiểm tra liên thông end-to-end 16
5.10 Kết quả kiểm thử tự động 17
5.11 Nhận xét về độ hội tụ và tính chịu lỗi 18

VI THẢO LUẬN **18**

6.1 So sánh Packet Tracer/IOS với GNS3/FRR 19
6.2 Các vấn đề gặp phải và cách xử lý 19
6.3 Hạn chế 19
6.4 Bài học kinh nghiệm 20

VII KẾT LUẬN **20**

7.1 Kết luận 20

A CẤU HÌNH FRR ĐẦY ĐỦ **21**
A.1 R1 21
A.2 R2 21
A.3 R3 22
B CẤU TRÚC THƯ MỤC VÀ LỆNH CHẠY **23**
C BẢNG KIỂM THỬ TỰ ĐỘNG **24**

I. TỔNG QUAN ĐỀ BÀI VÀ MỤC TIÊU

1.1 Đề bài

Thiết lập mô hình mạng gồm bốn router R0, R1, R2, R3 kết nối với nhau theo dạng vòng (ring topology). Trên tất cả router, cấu hình giao thức định tuyến động RIPv2 để bảo đảm liên thông toàn mạng. Tại Network A, triển khai một DHCP server để tự động cấp phát địa chỉ IP cho các máy trạm kết nối vào switch của mạng này. Bốn mạng LAN được đánh số như sau:

- Network A: `192.168.10.0/24`, gateway `192.168.10.1` (R0).
- Network B: `192.168.20.0/24`, gateway `192.168.20.1` (R1).
- Network C: `192.168.30.0/24`, gateway `192.168.30.1` (R2).
- Network D: `192.168.40.0/24`, gateway `192.168.40.1` (R3).

1.2 Mục tiêu bài lab

1. Thiết lập đúng sơ đồ kết nối vật lý và logic cho bốn router cùng bốn mạng LAN.
2. Gán địa chỉ IP chính xác cho tất cả interface của router, DHCP server và máy trạm.
3. Triển khai RIPv2 trên toàn bộ router, kiểm tra bảng định tuyến và trạng thái tiến trình RIP.
4. Triển khai DHCP server tại Network A, cấp phát động địa chỉ trong dải `192.168.10.10--192.168.10.100` với gateway và DNS là `192.168.10.1`.
5. Kiểm tra khả năng liên thông end-to-end bằng ping từ Network A sang gateway của các mạng còn lại và theo chiều ngược lại.
6. Tự động hóa việc tạo topology, cấu hình và kiểm thử bằng Python thông qua GNS3 API.

1.3 Phạm vi và lựa chọn triển khai

Báo cáo gốc thực hiện bài lab trên Cisco Packet Tracer với router mô phỏng của Packet Tracer và DHCP server dạng Server0. Báo cáo này giữ nguyên thiết kế về topology, địa chỉ IP, vai trò các node và kịch bản kiểm thử, nhưng thay đổi nền tảng triển khai:

- **GNS3 3.0.6** thay cho Packet Tracer: GNS3 cho phép chạy các node mạng thật (Docker, máy ảo, thiết bị phần cứng) và quản lý qua API.
- **FRRouting 8.4** trong Docker thay cho Cisco IOS: FRR là bộ định tuyến mã nguồn mở, hỗ trợ RIPv2 thật và có CLI `vtysh` gần với thao tác của IOS. Cách này không cần image Cisco IOSv có bản quyền.

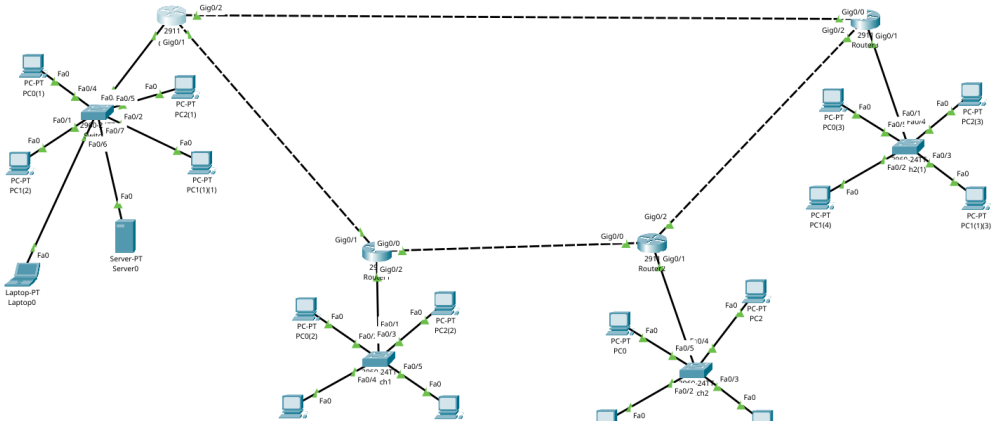
- **Alpine Linux + BusyBox `udhcpd`** thay cho Server0 của Packet Tracer.
- **VPCS 0.8.5** thay cho các Laptop mô phỏng, hỗ trợ cả DHCP và IP tĩnh.

Việc thay thế chỉ tác động đến công cụ mô phỏng, không làm thay đổi logic của bài lab. Đây cũng là nội dung chính được trình bày trong các chương sau.

II. SƠ ĐỒ MẠNG VÀ BẢNG PHÂN BỔ ĐỊA CHỈ IP

2.1 Sơ đồ mạng theo báo cáo gốc

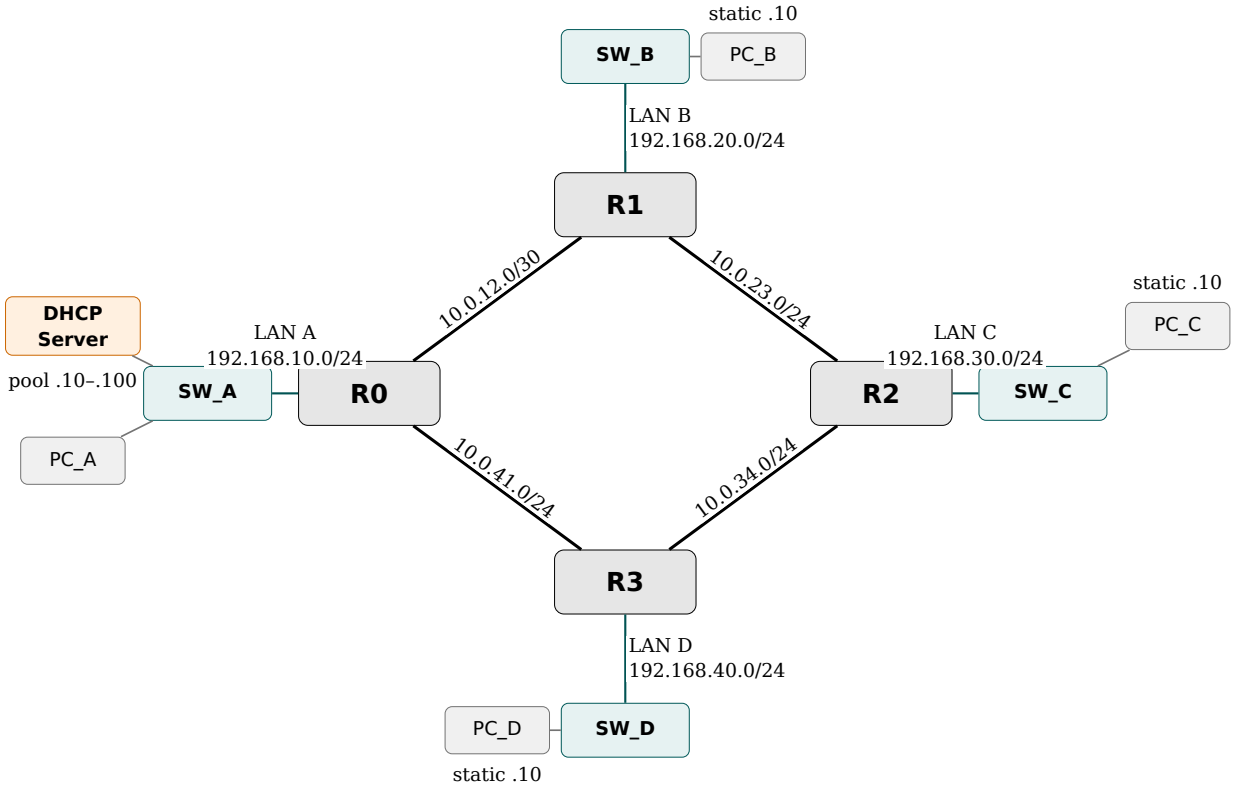
Hình 1 là sơ đồ mạng vòng bốn router được trích từ báo cáo PBL3 gốc. Sơ đồ thể hiện bốn router R0–R3 nối thành vòng, mỗi router quản lý một mạng LAN thông qua switch, và DHCP server nằm trong Network A.



Hình 1. Sơ đồ mạng vòng bốn router trong báo cáo gốc (Packet Tracer).

2.2 Sơ đồ triển khai trên GNS3

Hình 2 là sơ đồ triển khai thực tế trên GNS3. Bốn router FRR được đặt tại bốn đỉnh của vòng; mỗi router nối tới một switch LAN, từ switch nối tới máy trạm. DHCP server nối vào `SW_A` và có địa chỉ `192.168.10.2`. Các đường nét đậm là liên kết WAN giữa các router; các đường nét mảnh là liên kết LAN.



Hình 2. Sơ đồ triển khai trên GNS3: router FRR, switch LAN, PC và DHCP server.

2.3 Bảng liên kết WAN

Bảng 1 tóm tắt bốn liên kết WAN tạo thành vòng. Liên kết R0–R1 dùng subnet `/30` theo đúng báo cáo gốc; ba liên kết còn lại dùng `/24`.

Bảng 1. Các liên kết WAN trong mô hình.

Kết nối	Mạng WAN	Subnet mask	Ghi chú
R0 – R1	10.0.12.0/30	255.255.255.252	Liên kết trực tiếp, subnet nhỏ nhất
R1 – R2	10.0.23.0/24	255.255.255.0	Liên kết trực tiếp
R2 – R3	10.0.34.0/24	255.255.255.0	Liên kết trực tiếp
R3 – R0	10.0.41.0/24	255.255.255.0	Liên kết trực tiếp

2.4 Bảng phân bổ địa chỉ IP

Bảng 2 liệt kê địa chỉ IP trên từng interface. Tên interface được giữ theo cách đặt của FRR trong container (`eth0`, `eth1`, `eth2`); vai trò và thứ tự kết nối tương ứng với các cổng GigabitEthernet trong báo cáo gốc.

Bảng 2. Bảng phân bổ địa chỉ IP.

Thiết bị	Interface	Địa chỉ IP	Subnet mask	Kết nối / ghi chú
R0	eth0	192.168.10.1	255.255.255.0	Gateway Network A (SW_A)
	eth1	10.0.12.1	255.255.255.252	Kết nối R0 – R1
	eth2	10.0.41.2	255.255.255.0	Kết nối R0 – R3
R1	eth0	10.0.23.1	255.255.255.0	Kết nối R1 – R2
	eth1	192.168.20.1	255.255.255.0	Gateway Network B (SW_B)
	eth2	10.0.12.2	255.255.255.252	Kết nối R1 – R0
R2	eth0	10.0.23.2	255.255.255.0	Kết nối R2 – R1
	eth1	192.168.30.1	255.255.255.0	Gateway Network C (SW_C)
	eth2	10.0.34.1	255.255.255.0	Kết nối R2 – R3
R3	eth0	10.0.41.1	255.255.255.0	Kết nối R3 – R0
	eth1	192.168.40.1	255.255.255.0	Gateway Network D (SW_D)
	eth2	10.0.34.2	255.255.255.0	Kết nối R3 – R2
DHCP server	eth0	192.168.10.2	255.255.255.0	Gateway: 192.168.10.1

2.5 Dải địa chỉ DHCP

DHCP server chỉ phục vụ Network A. Cấu hình pool gồm: địa chỉ bắt đầu `192.168.10.10`, địa chỉ kết thúc `192.168.10.100`, gateway mặc định `192.168.10.1` và DNS `192.168.10.1`. Các máy PC_B, PC_C, PC_D dùng địa chỉ tĩnh lần lượt là `192.168.20.10`, `192.168.30.10`, `192.168.40.10` để bảo đảm kết quả kiểm thử ổn định.

III. CÔNG NGHỆ VÀ KIẾN TRÚC TRIỂN KHAI

3.1 GNS3 và API v3

GNS3 3.0.6 được chạy ở chế độ local server tại `http://127.0.0.1:3080`, sử dụng API v3. Mỗi project GNS3 là một tập hợp các node và link; các node Docker được GNS3 quản lý qua Docker engine trên cùng máy. Trong bài lab, compute “local” cung cấp cả ba loại node cần thiết: `docker`, `vpcs` và `ethernet_switch`.

Các thao tác tạo project, node, link và cấu hình console đều được thực hiện bằng Python thay vì thao tác thủ công trên GUI. Cách làm này bảo đảm lab có thể tái lập, đồng thời giảm sai sót khi phải tạo 13 node và 13 liên kết.

3.2 FRRouting trong Docker

Mỗi router là một container từ image `frrouting/frr:latest`, chạy FRRouting 8.4. Bên trong container, các tiến trình chính gồm `zebra` (quản lý interface và bảng định tuyến), `ripd` (tiến trình RIPv2) và `staticd` (route tĩnh). Cấu hình được đưa vào container qua `start_command` của GNS3; sau khi khởi động, có thể kiểm tra bằng `vtysh`.

RIPv2 trong bài lab được cấu hình với `version 2`, quảng bá các mạng LAN và WAN bằng câu lệnh `network`. RIPv2 là giao thức classless, gửi kèm subnet mask trong bản tin cập nhật; FRR 8.4 không cung cấp lệnh `no auto-summary` như IOS, nhưng hành vi classless vẫn được bảo đảm. Đây là điểm khác biệt cần lưu ý khi đối chiếu với báo cáo gốc.

Các thông số RIPv2 quan sát được trên R0: gửi cập nhật mỗi 30 giây (sai lệch $\pm 50\%$), timeout sau 180 giây, garbage collect sau 120 giây, gửi và nhận version 2 trên cả ba interface, administrative distance mặc định 120.

3.3 DHCP server

DHCP server là container Alpine Linux 3.20, chạy BusyBox `udhcpd` ở chế độ foreground. Cấu hình được ghi vào `/etc/udhcpd.conf` với interface `eth0`, pool `192.168.10.10--192.168.10.100`, gateway và DNS `192.168.10.1`. BusyBox `udhcpd` là lựa chọn nhẹ, không cần cài thêm gói và phù hợp với môi trường container.

3.4 Máy trạm VPCS

Bốn máy trạm dùng VPCS 0.8.5. PC_A được cấu hình `ip dhcp` để nhận địa chỉ động; PC_B, PC_C, PC_D được cấu hình IP tĩnh với gateway tương ứng. VPCS cung cấp các lệnh `show ip` và `ping`, đủ để kiểm tra cấu hình và liên thông mà không cần hệ điều hành đầy đủ.

3.5 Kiến trúc tự động hóa

Bộ script Python gồm các thành phần sau:

Bảng 3. Các script và vai trò trong bộ lab.

File	Vai trò
`doctor.py`	Kiểm tra GNS3 API, template, compute, Docker image, VPCS, Dynamips, `ubridge` và BusyBox trước khi chạy lab.
`setup.py`	Tạo project, template, 13 node, 13 link; gán IP và cấu hình RIPv2/DHCP; lưu `state.json`.
`test.py`	Kiểm tra bảng định tuyến, trạng thái RIP, ping từ bốn PC và DHCP client; trả mã lỗi nếu có hạng mục FAIL.
`cleanup.py`	Xóa project và template do lab tạo, có xác nhận trước khi xóa.
`common.py`	Hàm dùng chung: gọi API, load/save state, đường dẫn log.

Quy trình chạy lab gồm bốn bước: `./doctor.sh` kiểm tra môi trường; `./setup.sh` tạo topology và cấu hình; đợi khoảng 15 giây để RIPv2 và DHCP hội tụ; `./test.sh` kiểm thử tự động. Kết quả và log được lưu trong thư mục `logs/` và `state.json`.

IV. NỘI DUNG CẤU HÌNH

4.1 Cấu hình FRR cho R0

Đoạn mã 1 là cấu hình đầy đủ của R0. Cấu hình R1–R3 có cùng cấu trúc, chỉ khác địa chỉ IP và các mạng được quảng bá; toàn bộ được trình bày trong Phụ lục A.

Đoạn mã 1. Cấu hình FRR của R0.

```
hostname R0
!
interface eth0
 ip address 192.168.10.1/24
 no shutdown
 exit
!
interface eth1
 ip address 10.0.12.1/30
 no shutdown
 exit
!
interface eth2
 ip address 10.0.41.2/24
 no shutdown
 exit
!
router rip
 version 2
 network 192.168.10.0/24
```

```
 network 10.0.12.0/30
 network 10.0.41.0/24
 exit
!
```

4.2 Tóm tắt cấu hình R1, R2, R3

Bảng 4 tóm tắt interface và các mạng được quảng bá bởi ba router còn lại.

Bảng 4. Interface và mạng quảng bá của R1–R3.

Router	LAN	Interface LAN	Mạng WAN và subnet
R1	192.168.20.0/24	eth1: 192.168.20.1/24	eth0: 10.0.23.1/24 (R1–R2); eth2: 10.0.12.2/30 (R1–R0)
R2	192.168.30.0/24	eth1: 192.168.30.1/24	eth0: 10.0.23.2/24 (R2–R1); eth2: 10.0.34.1/24 (R2–R3)
R3	192.168.40.0/24	eth1: 192.168.40.1/24	eth0: 10.0.41.1/24 (R3–R0); eth2: 10.0.34.2/24 (R3–R2)

4.3 Cấu hình DHCP server

Đoạn mã 2 là nội dung `/etc/udhcpd.conf` trong container DHCP. Server lắng nghe trên `eth0`, cấp địa chỉ trong dải `.10--.100`, đồng thời cung cấp gateway và DNS là `192.168.10.1`.

Đoạn mã 2. Cấu hình BusyBox `udhcpd`.

```
start 192.168.10.10
end 192.168.10.100
interface eth0
opt router 192.168.10.1
opt dns 192.168.10.1
opt subnet 255.255.255.0
lease_file /tmp/udhcpd.leases
```

4.4 Cấu hình máy trạm VPCS

PC_A nhận cấu hình động; ba máy còn lại dùng IP tĩnh. Các lệnh tiêu biểu:

Đoạn mã 3. Lệnh cấu hình và kiểm tra trên VPCS.

```
# PC_A
ip dhcp
show ip

# PC_B
ip 192.168.20.10/24 192.168.20.1
show ip

```

```
# PC_C
ip 192.168.30.10/24 192.168.30.1

# PC_D
ip 192.168.40.10/24 192.168.40.1
```

4.5 Một số điểm kỹ thuật khi tự động hóa

- **Gửi `start_command` qua API v3:** trường này phải nằm trong `properties` của node Docker. Nếu gửi ở cấp ngoài cùng, GNS3 sẽ bỏ qua và container khởi động mà không có cấu hình.
- **Đặt tên container:** GNS3 tạo tên dạng `GNS3.<tên_node>.<project_id>`. Tên node `DHCP-Server` được giữ nguyên trong tên container; tên có dấu gạch dưới như `DHCP_Server` được xử lý tương ứng.
- **Console VPCS:** sau khi kết nối lại, VPCS có thể chưa in prompt ngay. Script gửi một ký tự xuống dòng trước khi `expect` prompt để tránh treo.
- **Kiểm tra router FRR:** image FRR không chạy telnet console, nên dùng `docker exec <container> vtysh -c "show ip route"` thay cho console GNS3.
- **Lệnh RIP:** FRR dùng `show ip rip status`; lệnh `show ip protocols` của IOS không có trong FRR.

V. KẾT QUẢ THỰC NGHIỆM

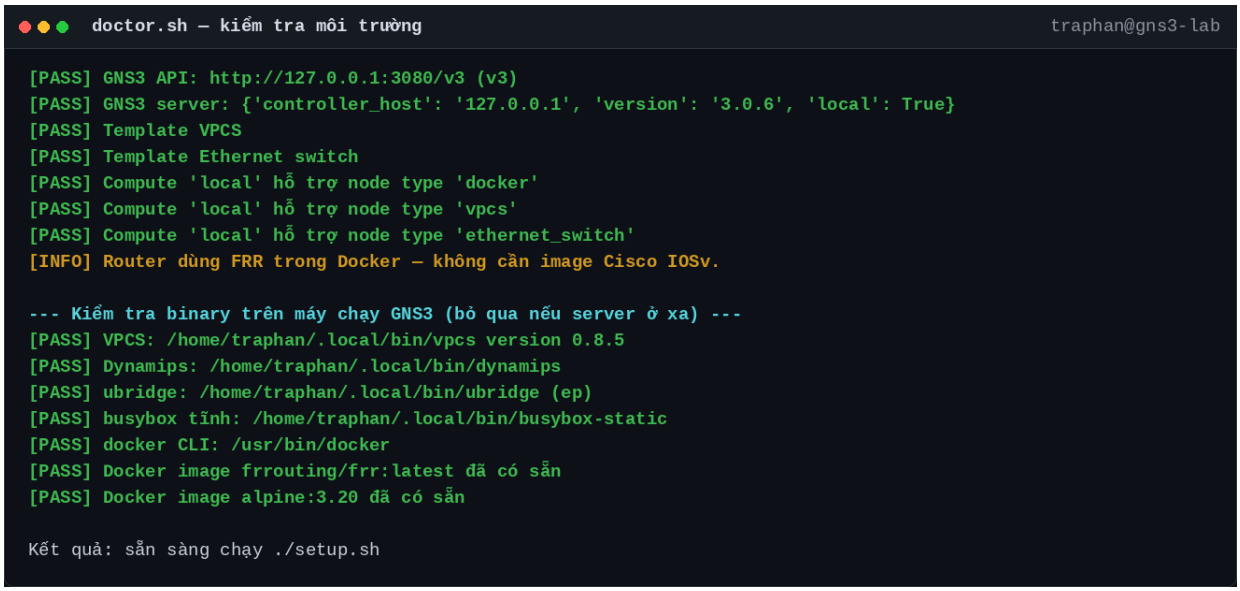
5.1 Môi trường thử nghiệm

Bảng 5. Môi trường chạy lab.

Thành phần	Phiên bản	Ghi chú
Hệ điều hành	Debian GNU/Linux 13 (trixie)	Kernel 6.12.111+deb13-amd64
GNS3	3.0.6	Local server, API v3
FRRouting	8.4_git	Container `frrouting/frr:latest`
Docker	29.9.2	Chạy trên cùng máy
DHCP server	Alpine 3.20 + BusyBox	`udhcpd`
VPCS	0.8.5	Bốn máy trạm
Python	3.13.5	Script tự động hóa

5.2 Kiểm tra tiền đề

Hình 3 là kết quả `./doctor.sh`. Tất cả hạng mục đều PASS: GNS3 API, template VPCS, template Ethernet switch, compute hỗ trợ Docker/VPCS, các binary VPCS, Dynamips, `ubridge`, BusyBox tĩnh, Docker CLI và hai image `frrouting/frr` và `alpine:3.20` đã có sẵn.



Hình 3. Kết quả `doctor.sh`: môi trường sẵn sàng chạy lab.

5.3 Khởi tạo topology

Hình 4 là kết quả `./setup.sh`. Script tạo project `PBL3-RIPv2-DHCP-FRR`, 4 router Docker, 4 switch, 4 PC VPCS và 1 DHCP server, sau đó tạo 13 liên kết đúng với sơ đồ. Bước cuối script chờ 15 giây để RIPv2 và DHCP hội tụ rồi gọi kiểm thử tự động.

setup.sh — dựng topology trên GNS3 traphan@gns3-lab

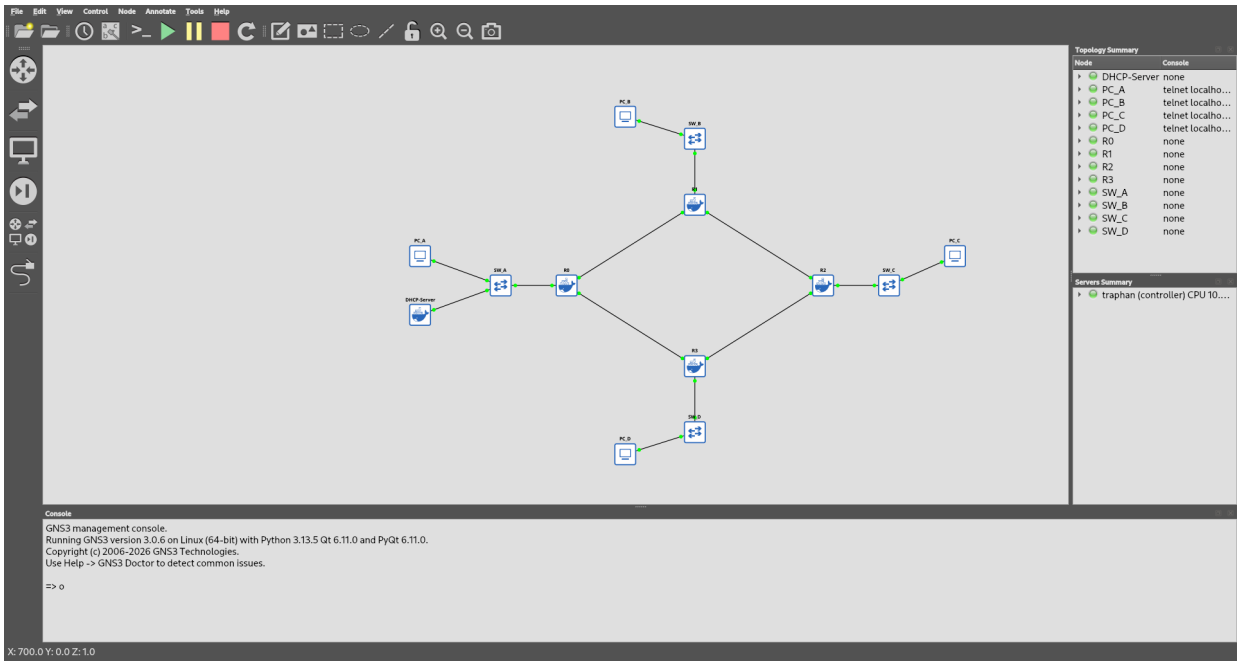
```
GNS3 API: http://127.0.0.1:3080/v3 (v3)
GNS3 server: {'controller_host': '127.0.0.1', 'version': '3.0.6', 'local': True}
Đã tạo project: PBL3-RIPv2-DHCP-FRR (4fe80004-eea4-4cb8-8b28-5ab7814a47c2)
  + R0 (docker)
  + R1 (docker)
  + R2 (docker)
  + R3 (docker)
  + SW_A (ethernet_switch)
  + SW_B (ethernet_switch)
  + SW_C (ethernet_switch)
  + SW_D (ethernet_switch)
  + PC_A (vpcs)
  + PC_B (vpcs)
  + PC_C (vpcs)
  + PC_D (vpcs)
  + DHCP_Server (docker)
  link R0:eth1 <-> R1:eth2
  link R1:eth0 <-> R2:eth0
  link R2:eth2 <-> R3:eth2
  link R3:eth0 <-> R0:eth2
  link R0:eth0 <-> SW_A:Ethernet0
  link R1:eth1 <-> SW_B:Ethernet0
  link R2:eth1 <-> SW_C:Ethernet0
  link R3:eth1 <-> SW_D:Ethernet0
  link SW_A:Ethernet1 <-> PC_A:Ethernet0
  link SW_A:Ethernet2 <-> DHCP_Server:eth0
  link SW_B:Ethernet1 <-> PC_B:Ethernet0
  link SW_C:Ethernet1 <-> PC_C:Ethernet0
  link SW_D:Ethernet1 <-> PC_D:Ethernet0
  Tất cả router FRR đã chạy ripd.
Cấu hình client VPCS...
Đợi RIP và DHCP hội tụ (15s)...
Chạy kiểm thử tự động...

Hoàn tất. Project ID: 4fe80004-eea4-4cb8-8b28-5ab7814a47c2
Mở GNS3 project: PBL3-RIPv2-DHCP-FRR
Log nằm trong: /home/traphan/Desktop/pbl3_gns3_ripv2_dhcp/logs
```

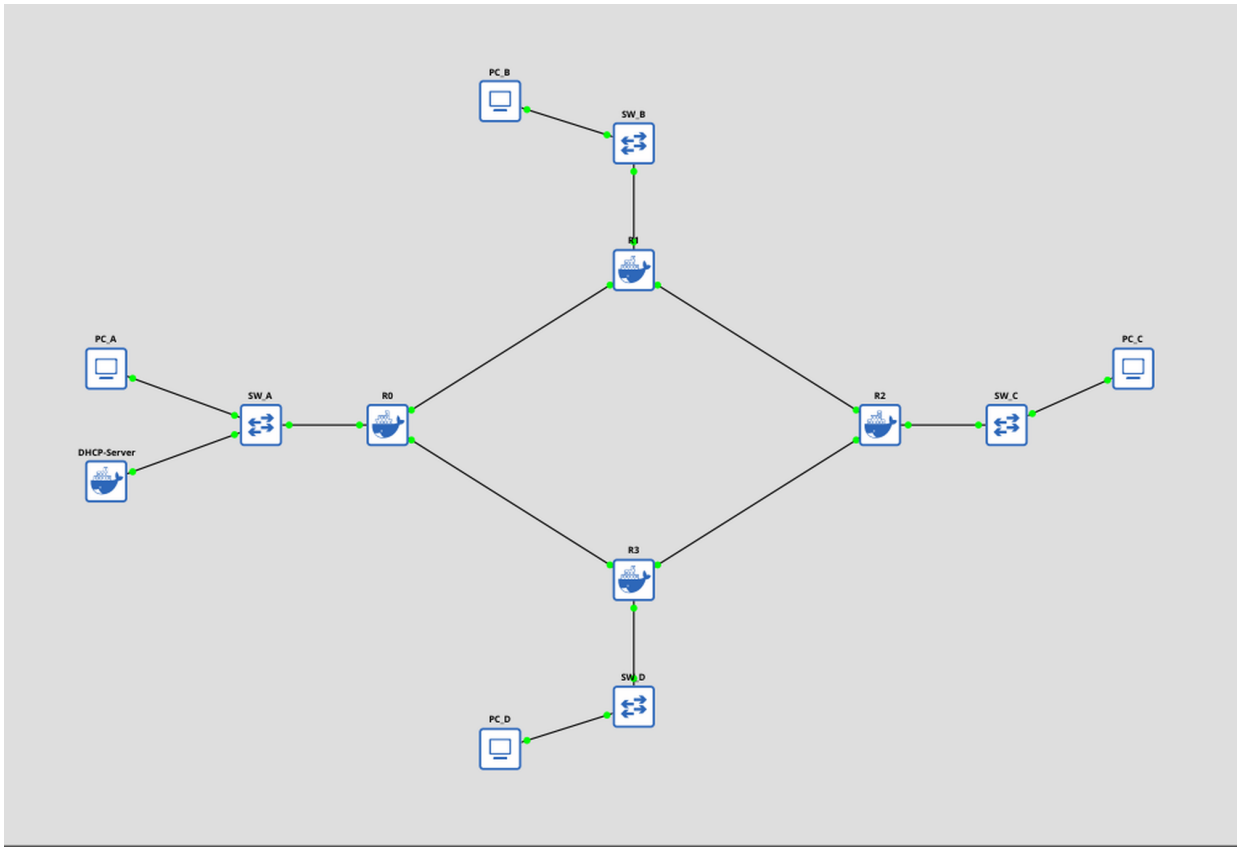
Hình 4. Quá trình `setup.sh` tạo project, node và link.

5.4 Giao diện GNS3

Hình 5 chụp cửa sổ GNS3 đang mở project. Phía trên là thanh công cụ; bên phải là danh sách node; phần giữa là canvas topology; phía dưới là console quản lý của GNS3. Hình 6 là ảnh cắt cận cảnh vùng topology để thấy rõ bốn router, bốn switch, bốn PC và DHCP server.



Hình 5. Giao diện GNS3 3.0.6 với project `PBL3-RIPv2-DHCP-FRR`.



Hình 6. Cận cảnh topology đang chạy trong GNS3.

5.5 Container của lab

Hình 7 liệt kê các container đang chạy. Bốn router dùng image `frrouting/frr`, DHCP server dùng `alpine:3.20`. Tên container chứa project ID để tránh trùng với các project GNS3 khác.

```
docker ps — container của lab                                    traphan@gns3-lab

DHCP-Server.4fe80004-eea4-4cb8-8b28-5ab7814a47c2    alpine:3.20    Up 30 minutes
R3.4fe80004-eea4-4cb8-8b28-5ab7814a47c2    frrouting/frr:latest    Up 30 minutes
R2.4fe80004-eea4-4cb8-8b28-5ab7814a47c2    frrouting/frr:latest    Up 30 minutes
R1.4fe80004-eea4-4cb8-8b28-5ab7814a47c2    frrouting/frr:latest    Up 30 minutes
R0.4fe80004-eea4-4cb8-8b28-5ab7814a47c2    frrouting/frr:latest    Up 30 minutes
```

Hình 7. Danh sách container Docker của lab.

5.6 Bảng định tuyến RIPv2

Sau khi hội tụ, R0 có đầy đủ route trực tiếp (C) và route học qua RIP (R). Hình 8 là kết quả `show ip route` trên R0. R0 học `10.0.23.0/24` và `192.168.20.0/24` qua R1, học `10.0.34.0/24` và `192.168.40.0/24` qua R3; `192.168.30.0/24` đi qua R1 với metric 3.

```
R0 — bảng định tuyến RIPv2                                       traphan@gns3-lab

R0# show ip route
Codes: K - kernel route, C - connected, S - static, R - RIP,
       O - OSPF, I - IS-IS, B - BGP, E - EIGRP, N - NHRP,
       T - Table, v - VNC, V - VNC-Direct, A - Babel, F - PBR,
       f - OpenFabric,
       > - selected route, * - FIB route, q - queued, r - rejected, b - backup
       t - trapped, o - offload failure

C>* 10.0.12.0/30 is directly connected, eth1, 00:30:27
R>* 10.0.23.0/24 [120/2] via 10.0.12.2, eth1, weight 1, 00:30:23
R>* 10.0.34.0/24 [120/2] via 10.0.41.1, eth2, weight 1, 00:30:19
C>* 10.0.41.0/24 is directly connected, eth2, 00:30:27
C>* 192.168.10.0/24 is directly connected, eth0, 00:30:27
R>* 192.168.20.0/24 [120/2] via 10.0.12.2, eth1, weight 1, 00:30:23
R>* 192.168.30.0/24 [120/3] via 10.0.12.2, eth1, weight 1, 00:30:21
R>* 192.168.40.0/24 [120/2] via 10.0.41.1, eth2, weight 1, 00:30:19
```

Hình 8. Bảng định tuyến trên R0 sau khi RIPv2 hội tụ.

Bảng 6 tổng hợp các route LAN học được trên cả bốn router. Ký hiệu “C” là kết nối trực tiếp; “R” là route học qua RIP, kèm next-hop và metric.

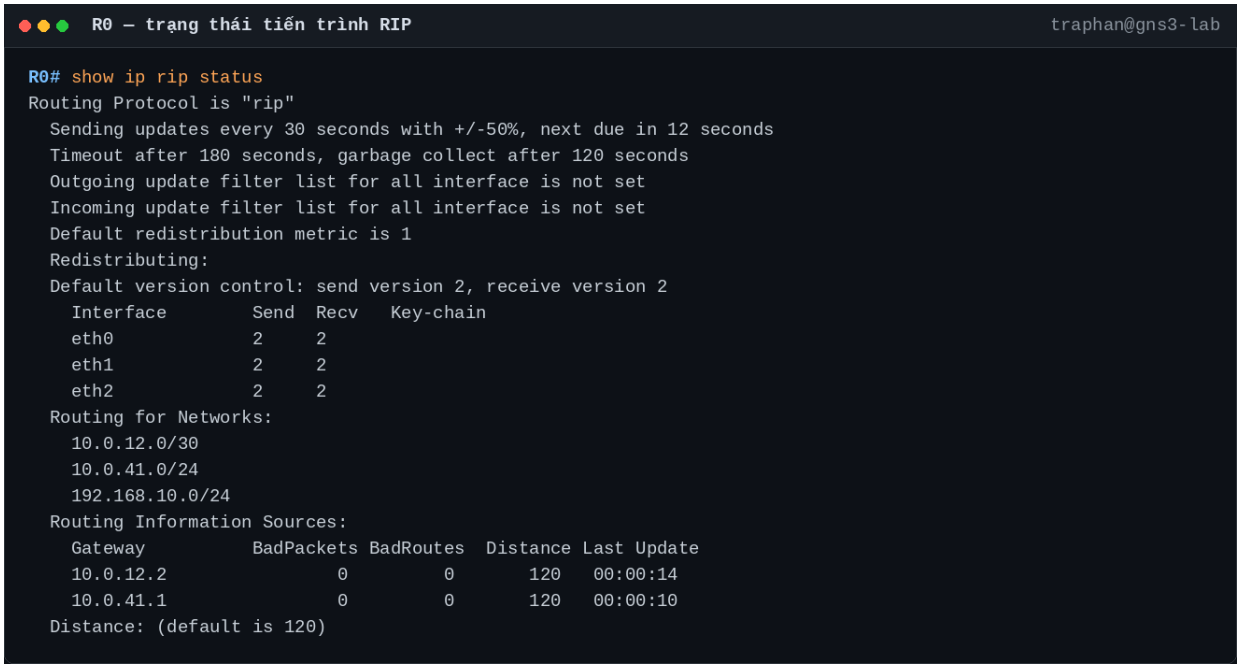
Bảng 6. Các mạng LAN trong bảng định tuyến của bốn router.

Router	192.168.10.0/24	192.168.20.0/24	192.168.30.0/24	192.168.40.0/24
R0	C	R, 10.0.12.2 (2)	R, 10.0.12.2 (3)	R, 10.0.41.1 (2)
R1	R, 10.0.12.1 (2)	C	R, 10.0.23.2 (2)	R, 10.0.23.2 (3)
R2	R, 10.0.23.1 (3)	R, 10.0.23.1 (2)	C	R, 10.0.34.2 (2)
R3	R, 10.0.41.2 (2)	R, 10.0.41.2 (3)	R, 10.0.34.1 (2)	C

5.7 Trạng thái tiến trình RIP

Hình 9 là kết quả `show ip rip status` trên R0. Tiến trình RIP gửi cập nhật mỗi 30 giây, timeout 180 giây, garbage collect 120 giây, gửi/nhận version 2 trên cả ba interface. Hai nguồn cập nhật là `10.0.12.2` (R1) và `10.0.41.1` (R3), không

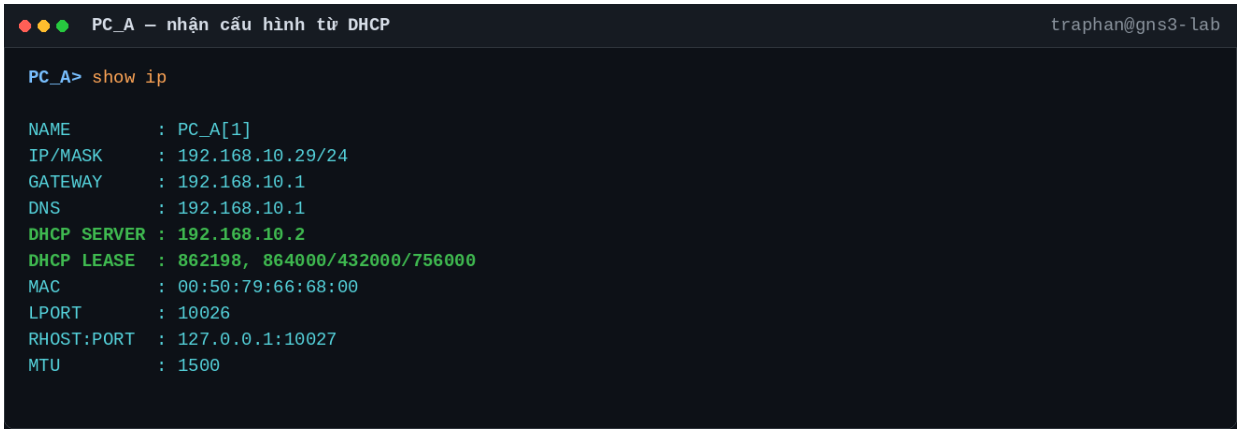
có bản tin lỗi (BadPackets = 0, BadRoutes = 0), khoảng cách quản trị mặc định 120.



Hình 9. Trạng thái tiến trình RIP trên R0.

5.8 Kiểm tra DHCP

Hình 10 là kết quả `show ip` trên PC_A. Máy nhận địa chỉ `192.168.10.29/24`, gateway `192.168.10.1`, DNS `192.168.10.1` và DHCP server `192.168.10.2`. Địa chỉ này nằm trong pool cấu hình; giá trị cụ thể có thể khác nhau giữa các lần chạy lab.

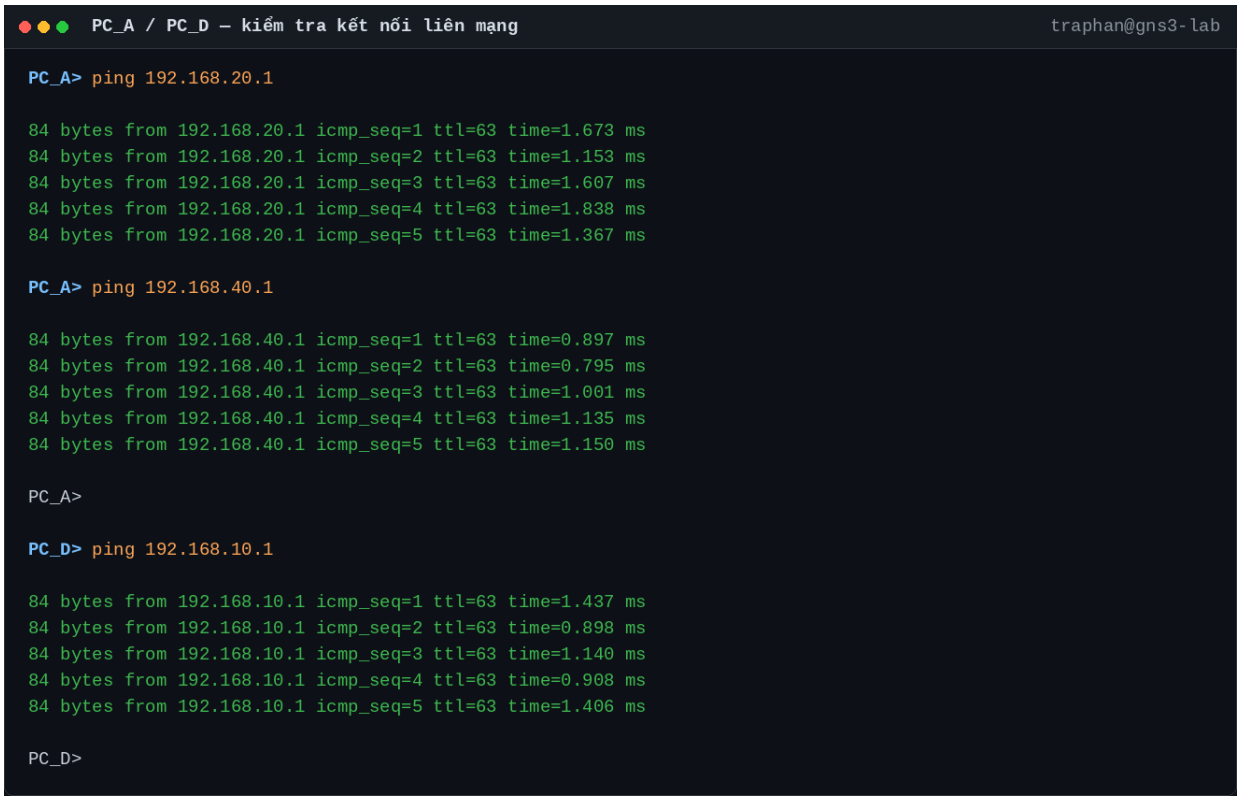


Hình 10. PC_A nhận cấu hình IP từ DHCP server.

5.9 Kiểm tra liên thông end-to-end

Hình 11 minh họa một số phép ping tiêu biểu từ PC_A và PC_D; Bảng 7 tổng hợp đầy đủ kết quả từ log, bao gồm cả gateway A và gateway C. Tất cả đều nhận 5/5 phản hồi. Giá trị TTL phản ánh số hop: PC_A tới `192.168.20.1` có TTL

63 (một router), tới `192.168.30.1` có TTL 62 (hai router), tới `192.168.40.1` có TTL 63 (một router theo nhánh R3).



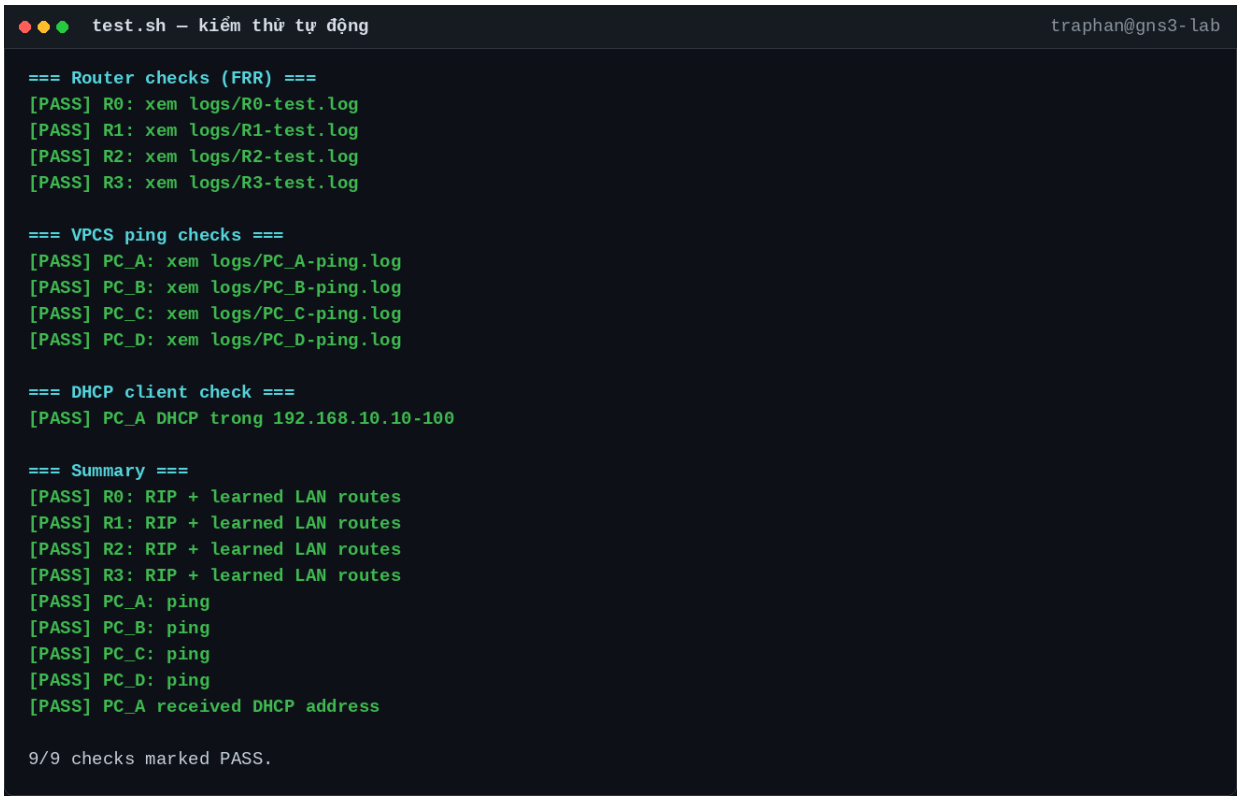
Hình 11. Ping liên mạng từ PC_A và PC_D.

Bảng 7. Tổng hợp một số kết quả ping tiêu biểu.

Nguồn	Đích	Kết quả	TTL
PC_A (192.168.10.29)	192.168.10.1 (GW A)	5/5, 100%	64
PC_A (192.168.10.29)	192.168.20.1 (GW B)	5/5, 100%	63
PC_A (192.168.10.29)	192.168.30.1 (GW C)	5/5, 100%	62
PC_A (192.168.10.29)	192.168.40.1 (GW D)	5/5, 100%	63
PC_D (192.168.40.10)	192.168.10.1 (GW A)	5/5, 100%	63

5.10 Kết quả kiểm thử tự động

Hình 12 là kết quả `./test.sh`. Bộ kiểm thử xác nhận bốn router có RIP và học đủ các mạng LAN, bốn PC ping được, và PC_A nhận địa chỉ DHCP hợp lệ. Kết quả cuối cùng là **9/9** hạng mục PASS.



Hình 12. Kết quả `test.sh`: 9/9 kiểm thử PASS.

5.11 Nhận xét về độ hội tụ và tính chịu lỗi

Sau khoảng 15 giây, cả bốn router đã có đủ route và không còn hiện tượng mất gói trong các phép ping. Trạng thái RIP cho thấy các bản tin cập nhật gần nhất cách thời điểm kiểm tra 10–14 giây, phù hợp với chu kỳ 30 giây. Với topology vòng, nếu một liên kết WAN bị đứt, RIPv2 có thể học đường thay thế qua nhánh còn lại; tuy nhiên trong phạm vi báo cáo này chưa thực hiện kịch bản phá liên kết, nên chưa có số liệu thời gian hội tụ khi có sự cố.

VI. THẢO LUẬN

6.1 So sánh Packet Tracer/IOS với GNS3/FRR

Bảng 8. So sánh hai cách triển khai.

Tiêu chí	Packet Tracer (báo cáo gốc)	GNS3 + FRR (báo cáo này)
Bản chất	Mô phỏng thiết bị và CLI Cisco	Router thật chạy FRR trong container Docker
Giấy phép	Cần image IOS/IOSv có bản quyền nếu muốn IOS thật	FRR mã nguồn mở, không cần image Cisco
CLI	Cisco IOS	`vtysh`, cú pháp gần IOS nhưng không giống hoàn toàn
DHCP	Server0 trên GUI	Alpine + BusyBox `udhcpd`
Máy trạm	Laptop mô phỏng	VPCS 0.8.5
Khả năng tái lập	Thao tác GUI, khó tự động hóa	Toàn bộ tạo bằng API/Python, có `state.json`
Độ chân thực	Mô phỏng	Gần với môi trường Linux/container thật

6.2 Các vấn đề gặp phải và cách xử lý

Bảng 9. Một số lỗi và cách khắc phục trong quá trình triển khai.

Vấn đề	Nguyên nhân	Cách xử lý
Router không nhận cấu hình	API v3 yêu cầu `start_command` nằm trong `properties`	Gửi `{"properties": {"start_command": ...}}`
VPCS treo khi kết nối lại	Prompt chưa được in sau khi reconnect	Gửi `\r\n` trước khi `expect(">")`
Lỗi `no auto-summary`	FRR 8.4 không hỗ trợ lệnh này	Bỏ lệnh; RIPv2 mặc định classless
Không có `show ip protocols`	FRR dùng lệnh khác	Dùng `show ip rip status`
Thiếu image IOS	IOSv cần bản quyền	Thay bằng FRR Docker
Không thấy file lease	BusyBox giữ lease trong bộ nhớ	Kiểm tra từ client bằng `show ip`

6.3 Hạn chế

- Router FRR không có console telnet trong GNS3; việc kiểm tra phải qua `docker exec` và `vtysh`, kém trực quan hơn console IOS.
- DHCP lease không được ghi ra file ổn định; khi container DHCP khởi động lại, lease cũ không còn. Với lab nhỏ, điều này không ảnh hưởng kết quả.

- Chưa kiểm thử phá liên kết WAN để đo thời gian hội tụ và kiểm chứng khả năng chịu lỗi của vòng.
- Chưa cấu hình xác thực RIPv2, route filtering, DHCP relay cho các LAN khác hoặc IPv6.
- Ảnh chụp GUI ở chế độ fit view có nhãn node nhỏ; sơ đồ TikZ được dùng để bổ sung phần trình bày trực quan.

6.4 Bài học kinh nghiệm

Việc chuyển một bài lab từ Packet Tracer sang GNS3 cho thấy khác biệt lớn nhất không nằm ở topology hay địa chỉ IP, mà ở cách kiểm chứng. Với thiết bị thật, mỗi router có tiến trình riêng, log riêng và có thể kiểm tra bằng CLI; do đó cần tự động hóa việc thu thập bằng chứng (bảng định tuyến, trạng thái RIP, lease DHCP, ping) thay vì chỉ nhìn vào giao diện mô phỏng. Bên cạnh đó, việc dùng API giúp tái lập lab nhanh, nhưng đòi hỏi hiểu rõ cấu trúc dữ liệu của GNS3, đặc biệt là các trường trong `properties` của node Docker.

VII. KẾT LUẬN

7.1 Kết luận

- Đã tái triển khai thành công bài lab PBL3 trên GNS3 với bốn router FRR, bốn switch, bốn PC và một DHCP server, giữ nguyên topology và địa chỉ IP của báo cáo gốc.
- RIPv2 hoạt động đúng: cả bốn router học đủ các mạng LAN, bảng định tuyến có đầy đủ route trực tiếp và route RIP, không có bản tin lỗi.
- DHCP server cấp phát thành công địa chỉ `192.168.10.29/24` cho PC_A, kèm gateway và DNS đúng cấu hình.
- Kiểm thử end-to-end đạt 100% cho các phép ping từ PC_A tới bốn gateway và từ PC_D về Network A; bộ kiểm thử tự động đạt 9/9.
- Bộ script Python và GNS3 API giúp toàn bộ quá trình tạo, kiểm tra và dọn dẹp lab có thể lặp lại, giảm phụ thuộc vào thao tác thủ công.

Tài liệu

[1] G. Malkin, *RIP Version 2*, RFC 2453, IETF, 1998.

[2] R. Droms, *Dynamic Host Configuration Protocol*, RFC 2131, IETF, 1997.

[3] FRRouting Project, *FRRouting Documentation – RIP*, `https://docs.frrouting.org/`.

[4] GNS3, *GNS3 Documentation – API v3*, `https://docs.gns3.com/`.

[5] BusyBox, *BusyBox udhcpd*, https://busybox.net/.

[6] GNS3, *Virtual PC Simulator (VPCS)*, https://github.com/GNS3/vpcs.

[7] Nhóm PBL3, *Báo cáo PBL3: Định tuyến động RIPv2 và máy chủ DHCP trên Cisco Packet Tracer*, 2026.

[8] Cisco Systems, *IP Routing: RIP Command Reference*, https://www.cisco.com/.

A. CẤU HÌNH FRR ĐẦY ĐỦ

Phần này trình bày cấu hình của R1, R2 và R3. Cấu hình R0 đã có ở Đoạn mã 1.

A.1 R1

Đoạn mã 4. Cấu hình FRR của R1.

```
hostname R1
!
interface eth0
 ip address 10.0.23.1/24
 no shutdown
 exit
!
interface eth1
 ip address 192.168.20.1/24
 no shutdown
 exit
!
interface eth2
 ip address 10.0.12.2/30
 no shutdown
 exit
!
router rip
 version 2
 network 10.0.23.0/24
 network 192.168.20.0/24
 network 10.0.12.0/30
 exit
!
```

A.2 R2

Đoạn mã 5. Cấu hình FRR của R2.

```
hostname R2
!
interface eth0
```

```
 ip address 10.0.23.2/24
 no shutdown
 exit
!
interface eth1
 ip address 192.168.30.1/24
 no shutdown
 exit
!
interface eth2
 ip address 10.0.34.1/24
 no shutdown
 exit
!
router rip
 version 2
 network 10.0.23.0/24
 network 192.168.30.0/24
 network 10.0.34.0/24
 exit
!
```

A.3 R3

Đoạn mã 6. Cấu hình FRR của R3.

```
hostname R3
!
interface eth0
 ip address 10.0.41.1/24
 no shutdown
 exit
!
interface eth1
 ip address 192.168.40.1/24
 no shutdown
 exit
!
interface eth2
 ip address 10.0.34.2/24
 no shutdown
 exit
!
router rip
 version 2
 network 10.0.41.0/24
 network 192.168.40.0/24
 network 10.0.34.0/24
 exit
!
```

B. CẤU TRÚC THƯ MỤC VÀ LỆNH CHẠY

Cấu trúc thư mục của lab:

Đoạn mã 7. Cấu trúc thư mục lab.

```
pbl3_gns3_ripv2_dhcp/
|-- setup.py                # create project, nodes, links, config
|-- test.py                 # automated tests
|-- doctor.py               # environment checks
|-- cleanup.py              # remove project
|-- setup.sh                # wrapper for setup.py
|-- test.sh                 # wrapper for test.py
|-- doctor.sh               # wrapper for doctor.py
|-- configs-frr/            # R0.conf ... R3.conf
|-- scripts/common.py       # shared helpers
|-- logs/                   # test logs
|-- state.json              # project id, console ports, containers
`-- report/                 # LaTeX report and images
```

Các lệnh chạy lab:

Đoạn mã 8. Quy trình chạy lab.

```
./doctor.sh              # check environment
./setup.sh               # create topology and configure
# wait about 15 seconds for RIPv2 and DHCP convergence
./test.sh                # automated tests
./cleanup.sh             # remove project when done
```

Kiểm tra router FRR trực tiếp:

Đoạn mã 9. Kiểm tra router qua `docker exec`.

```
docker exec GNS3.R0.<project-id> vtysh -c "show ip route"
docker exec GNS3.R0.<project-id> vtysh -c "show ip rip status"
```

C. BẢNG KIỂM THỬ TỰ ĐỘNG

Bảng 10. Danh sách 9 hạng mục kiểm thử.

STT	Hạng mục	Nội dung kiểm tra	Kết quả
1	R0	Có tiến trình RIP và đủ route LAN 20/30/40	PASS
2	R1	Có tiến trình RIP và đủ route LAN 10/30/40	PASS
3	R2	Có tiến trình RIP và đủ route LAN 10/20/40	PASS
4	R3	Có tiến trình RIP và đủ route LAN 10/20/30	PASS
5	PC_A	Ping được bốn gateway	PASS
6	PC_B	Ping được gateway B và A	PASS
7	PC_C	Ping được gateway C và A	PASS
8	PC_D	Ping được gateway D và A	PASS
9	DHCP	PC_A nhận IP trong dải 192.168.10.10–100	PASS